

Control Implementation for a Safe and Secure Self-Driving Car

PFE P2026

Quang Thinh BUI

PhD candidate Cong Khanh DINH
Assoc. Prof. Ionela PRODAN

LCIS – Grenoble INP, Université Grenoble Alpes

21 September 2026

Table of Contents

- ➊ Introduction
- ➋ Overview of the QCar2
- ➌ Trajectory Generation
- ➍ Controller Design
- ➎ Simulations and Experiments
- ➏ Conclusions and Future Directions

Table of Contents

① Introduction

- Motivation
- Challenges

② Overview of the QCar2

③ Trajectory Generation

④ Controller Design

⑤ Simulations and Experiments

⑥ Conclusions and Future Directions

Motivation

- **Safety** is ensured by enforcing vehicle state and input constraints not only at the current instant, but also over the future prediction horizon.
- **Computational efficiency** is essential to solve the optimization problem fast and reliably enough for real-time embedded implementation under limited computational and memory resources.

Our main goal is to design optimization-based control algorithms to ensure **safety and fast computation** in both simulations and experiments over an autonomous vehicle.



Challenges

- Working with a **new autonomous vehicle platform** Quanser QCar2 [Quanser \(2025\)](#).
- Reliability of sensor measurements.
- Integration with **Quanser Interactive Labs** [Quanser \(n.d.\)](#).
- Integration with **ROS2 Open Robotics (n.d.)** .
- Transition from simulation to experiments.
- Maintaining stable communication between the platform and the ground workstation.



Table of Contents

① Introduction

- ② Overview of the QCar2
- Hardware Description
 - Software Description
 - Computational Tools

③ Trajectory Generation

④ Controller Design

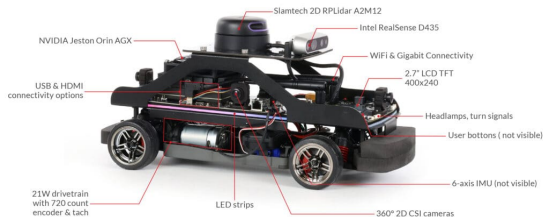
⑤ Simulations and Experiments

⑥ Conclusions and Future Directions

Hardware Description

- An open-architecture 1/10-scale autonomous vehicle equipped with an onboard compact computer, NVIDIA Jetson AGX Orin 32GB Quanser (2025).
- The car measures $0.39 \times 0.19 \times 0.20$ m, weighs 2.7 kg, and has a wheelbase of $l = 0.256$ m.
- The car is equipped with multiple sensors:
 - 2D LiDAR, RGB-D camera,
 - 4 CSI cameras providing 360° vision,
 - 6-axis IMU (3-axis gyroscope and 3-axis accelerometer),
 - 720-count motor encoder.

The QCar2 is connected via **SSH** to a ground workstation running Linux for real-time communication.



Software Description

Quanser Interactive Labs (QLabs)

- Simulation environment for the QCar2.
- Provides virtual models of the vehicle, sensors, and environment.
- Communication through the **Quanser Python API** for trajectory generation, tracking, and obstacle avoidance.



Robot Operating System 2 (ROS2)

- Communication framework for the physical QCar2.
- Distributed architecture based on **nodes and topics**, with control software implemented in C++ and Python.
- Receives vehicle and sensor data and publishes control commands.



Computational Tools

Gurobi

- Main optimization solver for the simulation studies.
- Used for NMPC, FLMP, and obstacle-avoidance MPC.
- Selected for its support of **mixed-integer optimization**, required by the OAMPC formulation.



acados

- Optimization framework used for the experimental NMPC and FLMP.
- Selected for efficient **real-time optimal control** on embedded hardware.
- **CasADi** provides symbolic modeling and solver generation, and the generated acados solver runs in a C++ controller within ROS2.



⁰NMPC (Nonlinear Model Predictive Control), FLMP (Flatness-based Model Predictive Control), OAMPC (Obstacle Avoidance Model Predictive Control)

Table of Contents

① Introduction

② Overview of the QCar2

③ Trajectory Generation

- Kinematic Model
- Flatness characterization
- Bézier Trajectory Generation

④ Controller Design

⑤ Simulations and Experiments

⑥ Conclusions and Future Directions

Kinematic Model

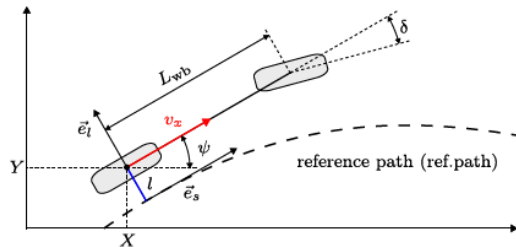
- Kinematic single-track model (Gratzer et al., 2023):

$$\dot{\mathbf{x}}(t) = \begin{bmatrix} \dot{X}(t) \\ \dot{Y}(t) \\ \dot{\psi}(t) \\ \dot{v}_x(t) \end{bmatrix} = \begin{bmatrix} v_x(t) \cos \psi(t) \\ v_x(t) \sin \psi(t) \\ \frac{v_x(t)}{L_{wb}} \tan \delta(t) \\ a_x(t) \end{bmatrix}.$$

- State and input vectors:

$$\mathbf{x} = [X \quad Y \quad \psi \quad v_x]^\top, \quad \mathbf{u} = [\delta \quad a_x]^\top.$$

- X, Y denote the position of the rear-axle center, ψ the heading angle, and v_x the longitudinal velocity.
- δ is the steering angle, a_x the longitudinal acceleration, and L_{wb} the length of the wheelbase.



Compatible with the QCar2 low-level interface using **velocity** and **steering commands**.

Model inversion through flatness

- Following the flatness-based formulation in [Gratzer et al. \(2023\)](#), the linear model tracks only the position coordinates X and Y . Therefore, ψ and v_x must be reconstructed to fully describe the desired vehicle trajectory.
- The flat output and its derivatives are

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix}, \quad \dot{\mathbf{z}} = \begin{bmatrix} \dot{z}_1 \\ \dot{z}_2 \end{bmatrix} = \begin{bmatrix} v_x \cos \psi \\ v_x \sin \psi \end{bmatrix}, \quad \ddot{\mathbf{z}} = \begin{bmatrix} a_x \cos \psi - \frac{v_x^2}{L_{wb}} \tan \delta \sin \psi \\ a_x \sin \psi + \frac{v_x^2}{L_{wb}} \tan \delta \cos \psi \end{bmatrix}.$$

- The vehicle states and inputs can be recovered from the flat variables:

$$\begin{aligned} X &= z_1, & Y &= z_2, \\ \psi &= \text{atan2}(\dot{z}_2, \dot{z}_1), & v_x &= \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \\ \delta &= \arctan \left(\frac{L_{wb}(\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1)}{(\dot{z}_1^2 + \dot{z}_2^2)^{3/2}} \right), & a_x &= \frac{\dot{z}_1 \ddot{z}_1 + \dot{z}_2 \ddot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}. \end{aligned}$$

- The vehicle and flat variables are related by: $(X_i, Y_i, \psi_i, v_{x,i}, \delta_i, a_{x,i}) \longleftrightarrow (\mathbf{z}_i, \dot{\mathbf{z}}_i, \ddot{\mathbf{z}}_i)$.

Bézier Trajectory Generation

- Each trajectory segment is defined by the initial and final flat boundary conditions:

$$\mathbf{z}(0), \dot{\mathbf{z}}(0), \ddot{\mathbf{z}}(0) \quad \text{and} \quad \mathbf{z}(T), \dot{\mathbf{z}}(T), \ddot{\mathbf{z}}(T).$$

- Using these boundary conditions, the trajectory is written as a fifth-order Bézier polynomial:

$$\mathbf{z}(s) = \sum_{i=0}^5 \mathbf{P}_i \binom{5}{i} (1-s)^{5-i} s^i, \quad s = \frac{t}{T} \in [0, 1].$$

- The six control points are obtained directly from the boundary conditions:

$$\mathbf{P}_0 = \mathbf{z}(0),$$

$$\mathbf{P}_5 = \mathbf{z}(T),$$

$$\mathbf{P}_1 = \mathbf{z}(0) + \frac{T}{5} \dot{\mathbf{z}}(0),$$

$$\mathbf{P}_4 = \mathbf{z}(T) - \frac{T}{5} \dot{\mathbf{z}}(T),$$

$$\mathbf{P}_2 = \mathbf{z}(0) + \frac{2T}{5} \dot{\mathbf{z}}(0) + \frac{T^2}{20} \ddot{\mathbf{z}}(0),$$

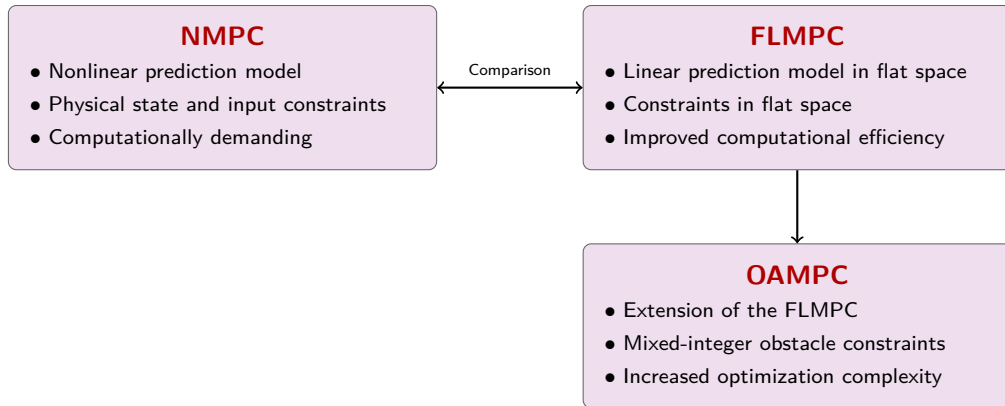
$$\mathbf{P}_3 = \mathbf{z}(T) - \frac{2T}{5} \dot{\mathbf{z}}(T) + \frac{T^2}{20} \ddot{\mathbf{z}}(T).$$

From the formulation above, a **reference trajectory** consistent with the QCar2 model is generated and used in the following simulations and experiments.

Table of Contents

- ① Introduction
- ② Overview of the QCar2
- ③ Trajectory Generation
- ④ Controller Design
 - Overview MPC Design
 - NMPC
 - FLMPc
 - OAMPC
- ⑤ Simulations and Experiments
- ⑥ Conclusions and Future Directions

Overview MPC Design



⁰NMPC (Nonlinear Model Predictive Control), FLMPC (Flatness-based Model Predictive Control), OAMPC (Obstacle Avoidance Model Predictive Control)

NMPC – Optimal Control Problem

- To minimize the tracking errors in the states and inputs over the prediction horizon N , the optimal control problem (OCP) is written as: [Tiriolo and Lucia \(2024\)](#)

$$\min_{\substack{\mathbf{x}_0, \dots, \mathbf{x}_N \\ \mathbf{u}_0, \dots, \mathbf{u}_{N-1}}} \sum_{i=0}^{N-1} \left[(\mathbf{x}_i - \mathbf{x}_{\text{ref},i})^\top Q (\mathbf{x}_i - \mathbf{x}_{\text{ref},i}) + (\mathbf{u}_i - \mathbf{u}_{\text{ref},i})^\top R (\mathbf{u}_i - \mathbf{u}_{\text{ref},i}) \right]$$

$$+ (\mathbf{x}_N - \mathbf{x}_{\text{ref},N})^\top P (\mathbf{x}_N - \mathbf{x}_{\text{ref},N})$$

$$\text{s.t. } \mathbf{x}_0 = \mathbf{x}_{\text{meas}}$$

$$\mathbf{x}_{i+1} = f_d(\mathbf{x}_i, \mathbf{u}_i) \quad i = 0, \dots, N-1$$

$$\mathbf{x}_i \in \mathcal{X} \quad i = 0, \dots, N$$

$$\mathbf{u}_i \in \mathcal{U} \quad i = 0, \dots, N-1$$

← Objective Function

← Initial Condition

← Nonlinear Model

← State Constraints

← Input Constraints

- The state and input constraint sets are defined as:

$$\mathcal{X} = \left\{ \mathbf{x} \in \mathbb{R}^4 : v_{x,\min} \leq v_x \leq v_{x,\max} \right\}, \quad \mathcal{U} = \left\{ \mathbf{u} \in \mathbb{R}^2 : \delta_{\min} \leq \delta \leq \delta_{\max}, a_{x,\min} \leq a_x \leq a_{x,\max} \right\}.$$

The discrete prediction model $f_d(\mathbf{x}_i, \mathbf{u}_i)$ is computed using the **fourth-order Runge–Kutta (RK4)** method.

FLMPC formulation

- For the FLMPC formulation [Gratzer et al. \(2023\)](#), the Cartesian flat output $\mathbf{y}$ and the corresponding flat state $\mathbf{z}$ are defined as:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix}, \quad \mathbf{z} = \begin{bmatrix} y_1 \\ \dot{y}_1 \\ y_2 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} X \\ v_x \cos \psi \\ Y \\ v_x \sin \psi \end{bmatrix} = t(\mathbf{x}).$$

- The virtual input is defined from the second derivatives of the flat outputs:

$$\boldsymbol{\nu} = \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \ddot{y}_1 \\ \ddot{y}_2 \end{bmatrix}.$$

- With sampling time T_s , the discrete flat-space model is:

$$\mathbf{z}_{k+1} = A_d \mathbf{z}_k + B_d \boldsymbol{\nu}_k,$$

$$A_d = \begin{bmatrix} 1 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & T_s \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad B_d = \begin{bmatrix} \frac{T_s^2}{2} & 0 \\ \frac{T_s}{2} & 0 \\ 0 & \frac{T_s^2}{2} \\ 0 & \frac{T_s}{2} \end{bmatrix}.$$

FLMPC – Input Constraints from the Original Space to the Flat Space

- Input constraints:

$$\mathcal{U} = \{\mathbf{u} : \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}\}, \quad \mathbf{u} = \begin{bmatrix} \delta \\ a_x \end{bmatrix}.$$

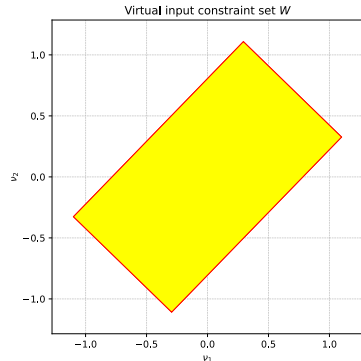
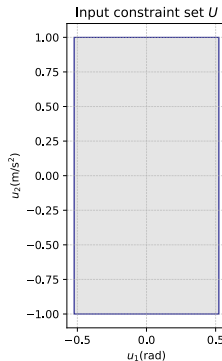
- Input constraints in the flat-output space:

$$\underbrace{\begin{bmatrix} -\frac{L_{wb}}{v_x^2} \sin \psi & \frac{L_{wb}}{v_x^2} \cos \psi \\ \cos \psi & \sin \psi \end{bmatrix}}_{M(\psi, v_x)} \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \tan \delta \\ a_x \end{bmatrix} = \tilde{\mathbf{u}}.$$

- Virtual input constraints:

$$W(v_x, \psi) = \{\nu : \tilde{\mathbf{u}}_{\min} \leq M(\psi, v_x)\nu \leq \tilde{\mathbf{u}}_{\max}\},$$

$$\tilde{\mathbf{u}}_{\min} = \begin{bmatrix} \tan \delta_{\min} \\ a_{x,\min} \end{bmatrix} \quad \tilde{\mathbf{u}}_{\max} = \begin{bmatrix} \tan \delta_{\max} \\ a_{x,\max} \end{bmatrix}.$$

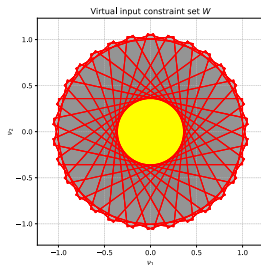


FLMPC – Worst-Case Virtual Input Constraints

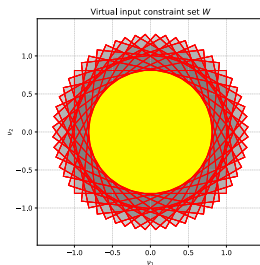
- A worst-case approximation is considered to obtain a single virtual-input constraint set over the prediction horizon by intersecting $W(v_x, \psi)$ over all heading angles for a fixed $v_x > 0$:

$$\widehat{W}(v_x) = \bigcap_{\psi \in [0, 2\pi)} W(v_x, \psi) = \{ \nu \in \mathbb{R}^2 : \nu_1^2 + \nu_2^2 \leq r^2(v_x) \}$$

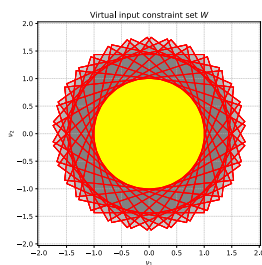
$$\text{where } r(v_x) = \min \left(a_{x, \max}, \frac{v_x^2}{L_{wb}} \tan \delta_{\max} \right), \quad \lim_{v_x \rightarrow 0^+} r(v_x) = 0.$$



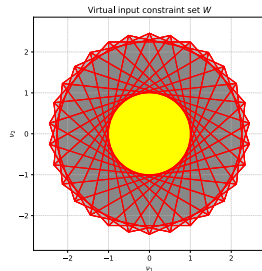
$$v_x = 0.4 \text{ m/s}$$



$$v_x = 0.6 \text{ m/s}$$



$$v_x = 0.8 \text{ m/s}$$



$$v_x = 1.0 \text{ m/s}$$

FLMPC – Optimal Control Problem

- The Cartesian position-tracking error is defined as:

$$\mathbf{e}_{k+j} = \begin{bmatrix} X_{k+j} \\ Y_{k+j} \end{bmatrix} - \begin{bmatrix} X_{\text{ref},k+j} \\ Y_{\text{ref},k+j} \end{bmatrix}.$$

- At each control step k , the FLMPC solves the following OCP over a prediction horizon N :

$$\begin{aligned} \mathbf{z}_k &= (\mathbf{z}_{k+1}, \dots, \mathbf{z}_{k+N}), \\ \mathbf{v}_k &= (\mathbf{v}_k, \dots, \mathbf{v}_{k+N-1}). \\ \min_{\mathbf{z}_k, \mathbf{v}_k} & \sum_{j=0}^{N-1} \left(\mathbf{e}_{k+j+1}^\top \mathbf{Q} \mathbf{e}_{k+j+1} + \mathbf{v}_{k+j}^\top \mathbf{R} \mathbf{v}_{k+j} \right) \\ \text{s.t.} \quad & \mathbf{z}_{k+j+1} = \mathbf{A}_d \mathbf{z}_{k+j} + \mathbf{B}_d \mathbf{v}_{k+j}, \quad j = 0, \dots, N-1, \\ & \mathbf{v}_{k+j} \in W(v_x, \psi), \quad j = 0, \dots, N-1. \end{aligned}$$

Remark on low-velocity sensitivity

- The optimal virtual input is converted back to the original physical input:

$$\mathbf{u}_k^* = \begin{bmatrix} \delta_k^* \\ a_{x,k}^* \end{bmatrix} = \begin{bmatrix} \arctan \left[\frac{L_{wb}}{v_x^2} \left(-v_{1,k}^* \sin \psi_k + v_{2,k}^* \cos \psi_k \right) \right] \\ v_{1,k}^* \cos \psi_k + v_{2,k}^* \sin \psi_k \end{bmatrix}.$$

With

$$\alpha = \frac{L_{wb}}{v_x^2}, \quad L_{wb} = 0.25725 \text{ m},$$

at $v_x = 0.5 \text{ m/s}$ and $v_x = 0.1 \text{ m/s}$, respectively:

$$\alpha \approx 1.029, \quad \alpha \approx 25.725.$$

At low velocity, a small variation of ψ or v_x may cause δ to reach its saturation limit.

OAMPC – Obstacle Avoidance Constraints

- A bounded convex polygonal obstacle $\mathcal{Q}$ with n_e edges is represented in the flat-output space as:

$$\mathcal{Q} = \left\{ \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} : G \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \leq \mathbf{f} \right\}, \quad \text{where } G \in \mathbb{R}^{n_e \times 2}, \mathbf{f} \in \mathbb{R}^{n_e \times 1}.$$

- To keep the predicted vehicle position outside $\mathcal{Q}$, at least one obstacle half-space inequality must be violated. Using the Big-M formulation:

$$\begin{aligned} f - G \begin{bmatrix} y_{1,k+j+1} \\ y_{2,k+j+1} \end{bmatrix} + (\gamma - s)\mathbf{1} &\leq M(\mathbf{1} - \alpha_{k+j+1}), \\ \mathbf{1}^\top \alpha_{k+j+1} &\geq 1, & j = 0, \dots, N-1, \\ \alpha_{k+j+1} &\in \{0, 1\}^{n_e}, & j = 0, \dots, N-1, \\ s &\geq 0. \end{aligned}$$

where M : big constant value,

γ : safety buffer,

s : slack variable,

α_{k+j+1} : binary vector for the n_e obstacle edges at predicted step $k + j + 1$,
with k the current step and j the prediction-horizon index.

OAMPC – Optimal Control Problem

- By extending the FLMPc with obstacle-avoidance constraints for n_{obs} obstacles, the OAMPc solves:

$$\begin{aligned}
&\min_{\mathbf{z}_k, \mathbf{v}_k, \boldsymbol{\alpha}, s} \sum_{j=0}^{N-1} \left(\mathbf{e}_{k+j+1}^\top Q \mathbf{e}_{k+j+1} + \mathbf{v}_{k+j}^\top R \mathbf{v}_{k+j} \right) + r_s s \\
&\text{s.t. } \mathbf{z}_{k+j+1} = A_d \mathbf{z}_{k+j} + B_d \mathbf{v}_{k+j} \\
&\quad \mathbf{v}_{k+j} \in W(\mathbf{v}_x, \psi) \\
&\quad \begin{cases} \mathbf{f}_m - G_m \begin{bmatrix} y_{1,k+j+1} \\ y_{2,k+j+1} \end{bmatrix} + (\gamma - s)\mathbf{1} \leq M(\mathbf{1} - \boldsymbol{\alpha}_{m,k+j+1}) \\ \mathbf{1}^\top \boldsymbol{\alpha}_{m,k+j+1} \geq 1, & \boldsymbol{\alpha}_{m,k+j+1} \in \{0, 1\}^{n_e, m} \\ s \geq 0 \end{cases}
\end{aligned}$$

Objective Function

Flat Model

Virtual-input constraints

Obstacle constraint

Binary Variables

Slack constraint

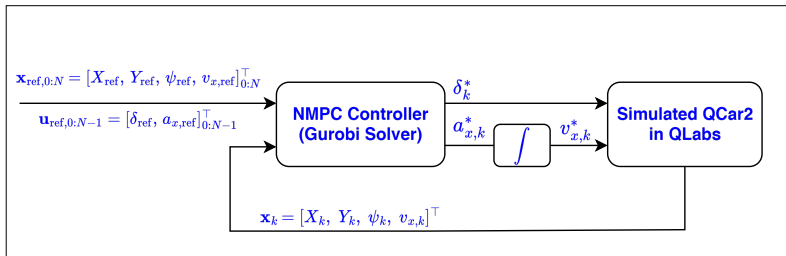
- The introduction of binary variables converts the FLMPFC formulation into a **Mixed-Integer Quadratic Programming (MIQP)** problem.

Table of Contents

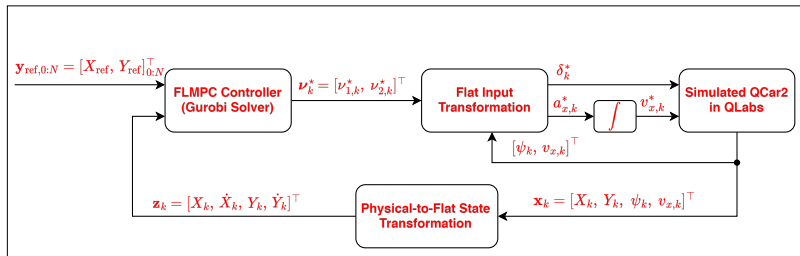
- ① Introduction
- ② Overview of the QCar2
- ③ Trajectory Generation
- ④ Controller Design
- ⑤ Simulations and Experiments
 - Simulation Results
 - Experiment Results
- ⑥ Conclusions and Future Directions

Simulation Control Schemes

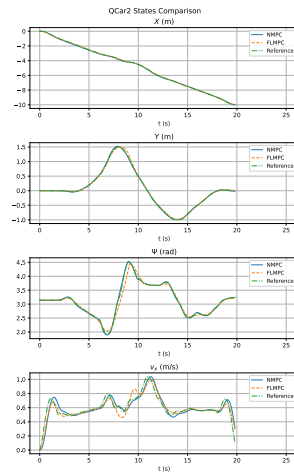
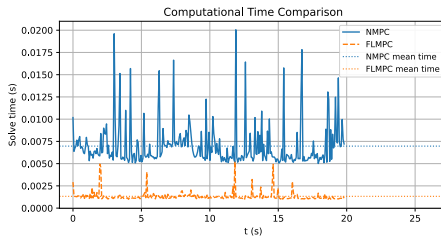
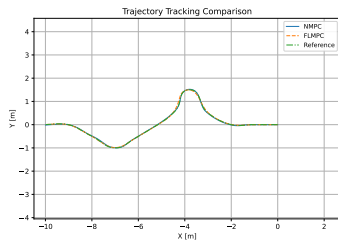
• NMPC:



• FL MPC:



NMPC vs FL MPC Simulation Comparison



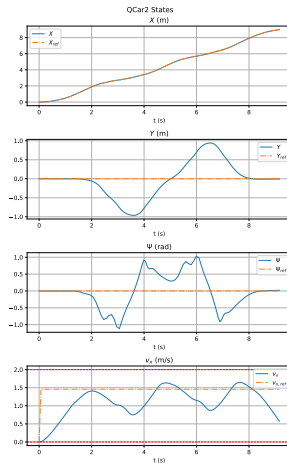
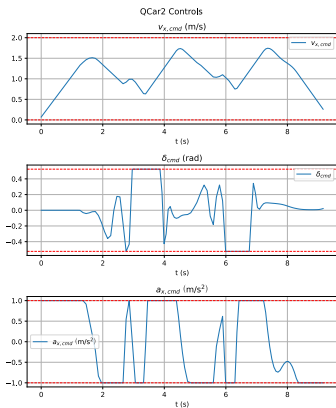
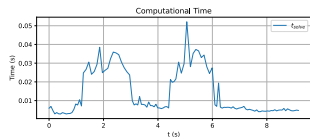
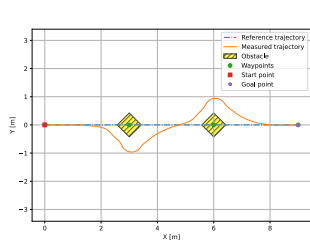
- With $N = 13$ and $T_s = 0.07$ s for both controllers.
- **NMPC:** $Q = \text{diag}(160, 160, 80, 200)$, $R = \text{diag}(10, 500)$, $P = \text{diag}(320, 320, 160, 400)$.
- **FL MPC:** $Q = \text{diag}(120, 120)$, $R = \text{diag}(5, 5)$.
- $-2.0 \leq v_x \leq 2.0$ m/s, $-0.5236 \leq \delta \leq 0.5236$ rad, $-1.0 \leq a_x \leq 1.0$ m/s².

Controller	RMSE position [m]	Maximum position error [m]	Mean solve time [s]
NMPC	0.0434	0.0910	0.0069
FL MPC	0.0345	0.1340	0.0013

State tracking

OAMPC Simulation Results

• OAMPC without obstacle activation:

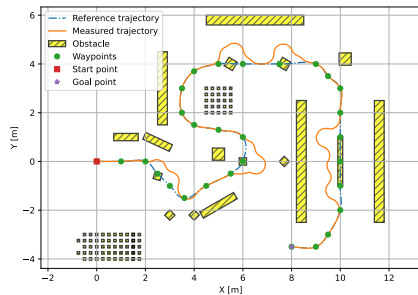


- The mean and maximum solve times are 0.0127 s and 0.0521 s, respectively.
- $N = 13$, $T_s = 0.07$ s, $Q = \text{diag}(160, 160)$ and $R = \text{diag}(2, 2)$.
- $0.0 \leq v_x \leq 2.0$ m/s, $-0.5236 \leq \delta \leq 0.5236$ rad, $-1.0 \leq a_x \leq 1.0$ m/s².

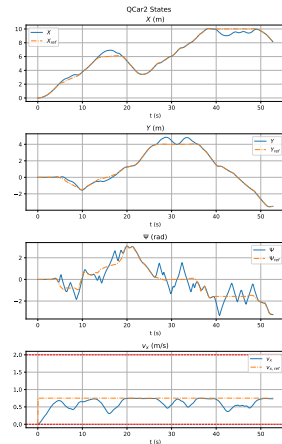
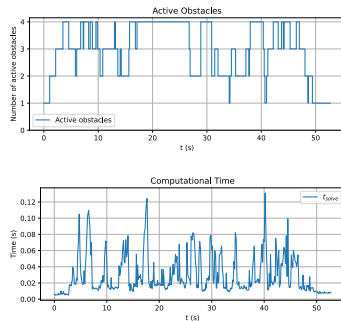
State tracking

OAMPC Simulation Results

• OAMPC in a multi-obstacle environment:



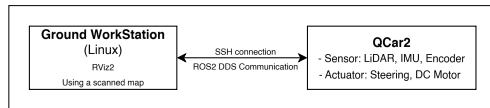
- The OCP weights and physical constraints are retained from the previous OAMPC simulation, while only N and T_s are changed to 30 and 0.02 s, respectively.
- The average number of active obstacles is 2.93, while the mean and maximum solve times are 0.0265 s and 0.1311 s, respectively.



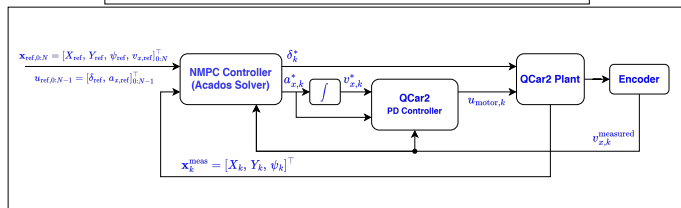
State tracking

Experiment Control Schemes

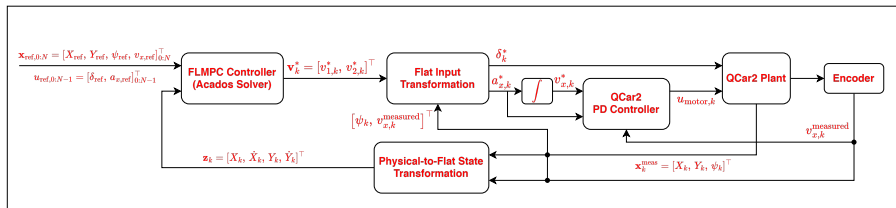
• Setup:



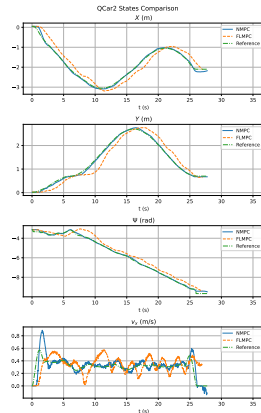
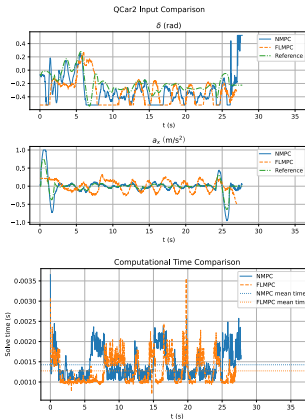
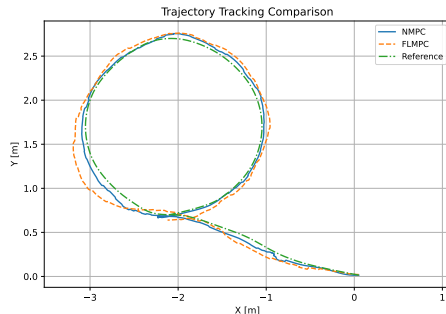
• NMPC:



• FL MPC:



NMPC and FL MPC Experiment Comparison



Controller	RMSE position [m]	Maximum position error [m]	Mean solve time [s]
NMPC	0.0691	0.2433	0.00143
FL MPC	0.0980	0.2673	0.00128

- $N = 30$ and $T_s = 0.03$ s for both controllers.
- NMPC: $Q = \text{diag}(160, 160, 80, 200)$, $R = \text{diag}(10, 500)$, $P = \text{diag}(320, 320, 160, 400)$.
- FL MPC: $Q = \text{diag}(120, 120)$, $R = \text{diag}(15, 15)$.

Table of Contents

- ① Introduction
- ② Overview of the QCar2
- ③ Trajectory Generation
- ④ Controller Design
- ⑤ Simulations and Experiments
- ⑥ Conclusions and Future Directions

Conclusions and Future Directions

Conclusions:

- The developed control framework was successfully validated in QLab simulation and transferred to the physical QCar2 platform.
- NMPC and FLMPc demonstrated successful trajectory tracking in both simulations and experiments while satisfying the vehicle constraints.
- We extended FLMPc to OAMPC for static-obstacle avoidance, successfully avoiding obstacles in simulation, but its computation time remains a limitation for real-time implementation and motivates further work on computational efficiency.

Future Directions:

- Reduce the OAMPC's computational burden to enable real-time obstacle-avoidance experiments.
- Investigate the CLF-based approach (Do et al., 2024), using the developed OAMPC as an offline reference controller to generate trajectories for constructing a lower-complexity online controller.

References

- [1] A. L. Gratzner, M. M. Broger, A. Schirrer, and S. Jakubek, "Flatness-Based Mixed-Integer Obstacle Avoidance MPC for Collision-Safe Automated Urban Driving," *CoDIT*, 2023.
- [2] C. Tiriolo and W. Lucia, "A Feedback Linearized Model Predictive Control Strategy for Input-Constrained Self-Driving Cars," *arXiv:2405.01753*, 2024.
- [3] Quanser, "QCar2," 2025.
- [4] Quanser, "QLabs Core Documentation."
- [5] Open Robotics, "Robot Operating System 2."
- [6] acados developers, "acados Documentation."
- [7] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual."
- [8] H.-T. Do, F. Blanchini, S. Miani, and I. Prodan, "LP-Generated Control Lyapunov Functions With Application to Multicopter Control," *IEEE Transactions on Control Systems Technology*, vol. 32, no. 6, pp. 2090–2101, 2024.
- [9] H.-T. Do, F. Blanchini, S. Miani, and I. Prodan, "Reducing Real-Time Complexity via Sub-Control Lyapunov Functions: From Theory to Experiments," *Automatica*, vol. 32, no. 6, pp. 2090–2101, 2025.

Thank you for your attention!